

SELF-LEARNING REPORT - 1

Agile, DevOps and Modern Requirements Engineering

Subject: Software Engineering and Architecture

Self-Learning Topic: Agile, DevOps and Advanced Requirement Engineering

Student Name: Jagdish Padhi **Class:** D12C **Year:** 2026-27

1. Introduction

Modern software development requires more than simply writing code. Successful software projects depend on effective requirements management, iterative development, collaboration between teams, continuous delivery, and constant feedback from stakeholders.

Traditional software development models often separated development, testing, deployment, and operations into different stages. Modern approaches such as **Agile** and **DevOps** attempt to reduce these boundaries and enable teams to deliver software faster and more reliably.

This self-learning study explores the relationship between Agile and DevOps, practical project-management tools such as **Jira**, techniques for writing effective **user stories and acceptance criteria**, advanced **requirements elicitation techniques**, the **Requirement Traceability Matrix (RTM)**, and modern requirement-management tools.

The concepts are also related to a practical software project scenario to understand how these techniques are applied beyond theory.

2. Learning Objectives

The major objectives of this self-learning activity are:

- Understand the principles and objectives of Agile and DevOps.
- Differentiate between Agile methodology and DevOps practices.
- Study how Jira supports Agile project management.
- Learn how to write clear and testable user stories.
- Understand acceptance criteria and their importance.
- Explore advanced requirement elicitation techniques.
- Understand the purpose and structure of a Requirement Traceability Matrix.
- Study tools used for professional requirement management.

3. Agile Software Development

Agile is an iterative approach to software development in which software is developed and delivered in small increments.

Instead of attempting to define and implement the entire system at the beginning of the project, Agile teams continuously develop, test, review, and improve the product.

The Agile approach emphasizes:

Customer collaboration → Working software → Continuous feedback → Adaptation to change

Key Characteristics of Agile

Characteristic	Description
Iterative development	Software is developed through repeated cycles called iterations or sprints.
Customer collaboration	Customers and stakeholders continuously provide feedback.
Adaptability	Requirements can change during development.
Frequent delivery	Working software is delivered regularly.
Team collaboration	Developers, testers, designers and stakeholders work closely together.
Continuous improvement	Teams review their processes and improve them after each iteration.

One of the most commonly used Agile frameworks is **Scrum**, where development work is organized into short iterations called **Sprints**.

4. DevOps

DevOps is a set of practices, principles, and cultural approaches that brings **software development (Development)** and **IT operations (Operations)** closer together.

The primary objective is to create a continuous software delivery process in which code can move efficiently from development to production.

A simplified DevOps lifecycle can be represented as:

Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → Feedback

DevOps commonly involves technologies such as version control systems, automated testing, CI/CD pipelines, containers, cloud platforms, infrastructure automation, and monitoring systems.

Benefits of DevOps

- Faster software delivery
- Automated testing and deployment
- Reduced manual errors
- Better collaboration between development and operations
- Faster detection and resolution of issues
- More reliable and repeatable deployments
- Continuous feedback from production systems

5. Agile vs DevOps

Agile and DevOps are closely related but they solve different aspects of software development.

Aspect	Agile	DevOps
Primary focus	Software development process	Software delivery and operations
Main objective	Deliver working software incrementally	Deliver and operate software continuously and reliably
Major concern	Requirements, development and feedback	Development, deployment, infrastructure and operations
Common practices	Sprints, Scrum, user stories, retrospectives	CI/CD, automation, monitoring, infrastructure as code
Feedback	Primarily from customers and stakeholders	From customers, systems, monitoring and operations
Automation	Helpful but not mandatory	Strong emphasis on automation
Teams	Development and business collaboration	Development, operations, QA and security collaboration

Relationship between Agile and DevOps

Agile can help a team **develop software in small, adaptive increments**, while DevOps helps the organization **build, test, deploy and operate those increments efficiently**.

Therefore, they should not be viewed as competing methodologies.

Agile improves the way software is developed; DevOps extends this process toward continuous delivery and operation.

6. Jira as an Agile Project Management Tool

Jira is a project-management and issue-tracking platform widely used by software development teams. It can be used to organize requirements, create tasks, assign responsibilities, track progress, manage bugs, and visualize the development workflow.

A typical Jira workflow can be represented as:

Backlog → To Do → In Progress → Code Review → Testing → Done

Important Jira Concepts

Epic:

A large feature or business requirement that can be divided into smaller tasks or user stories.

User Story:

A short description of a requirement from the perspective of an end user.

Task:

A specific piece of work that needs to be completed.

Bug:

An identified defect or unexpected behavior in the software.

Sprint:

A fixed development period during which a selected set of work is completed.

Backlog:

A prioritized collection of pending requirements, features, tasks and bugs.

Jira therefore acts as a bridge between requirements and actual development work.

7. Writing Effective User Stories

A user story describes a software requirement from the perspective of the person who will use the system.

A commonly used format is:

As a [user], I want [functionality], so that [benefit].

Example

As a student, I want to filter hackathons by deadline and technology so that I can quickly find relevant opportunities.

This statement identifies:

- **User:** Student
- **Requirement:** Filter hackathons
- **Benefit:** Quickly discover relevant opportunities

Characteristics of a Good User Story

A good user story should be:

- Clear and understandable
- Focused on user value
- Small enough to implement within an iteration
- Testable
- Independent where possible
- Negotiable rather than unnecessarily prescribing implementation details

8. Acceptance Criteria

Acceptance criteria define the conditions that must be satisfied for a user story to be considered complete.

For example, for the previous story:

User Story

As a student, I want to filter hackathons by deadline and technology so that I can quickly find relevant opportunities.

Acceptance Criteria

1. The user can select a deadline range.
2. The user can select one or more technologies.
3. Only hackathons satisfying the selected filters are displayed.
4. Applying filters does not remove unrelated hackathon information.
5. A suitable message is displayed when no matching hackathons are found.

Acceptance criteria make requirements **specific, measurable and testable**.

They also create a common understanding between the product owner, developer and tester.

9. Advanced Requirement Elicitation

Requirement elicitation is the process of discovering, understanding and documenting the needs of stakeholders. Simply asking stakeholders, "What features do you want?" may not always reveal the actual problem. Advanced elicitation techniques therefore attempt to understand both the **stated requirements** and the **underlying needs**.

9.1 Interviews

Interviews involve direct interaction with stakeholders to understand their goals, problems and expectations.

Questions can be:

- Open-ended: "What problems do you currently face?"
- Specific: "How do you currently track this information?"
- Follow-up: "Why is this process difficult?"

Interviews are useful when detailed domain knowledge is required.

9.2 Observation and Ethnography

In observation-based elicitation, analysts study how users actually perform their work in their real environment.

This can reveal problems that users may not explicitly mention during interviews.

For example, a user may say:

"The existing system works."

However, observing the process might reveal that the user manually copies data between multiple systems.

Ethnographic techniques are particularly useful for understanding complex real-world workflows.

9.3 Workshops

Requirement workshops bring multiple stakeholders together to discuss requirements collaboratively.

They are useful when different groups have different expectations.

For example:

Customer → Business Analyst → Developer → Tester → Operations Team

can collaboratively discuss a requirement and identify conflicts before implementation begins.

9.4 Surveys and Questionnaires

Surveys are useful when feedback needs to be collected from a large number of users. They provide quantitative information and can help identify common patterns or preferences.

9.5 Prototyping

A prototype is an early representation of the proposed system or feature. Instead of discussing a requirement only through text, stakeholders can interact with a visual representation. This can expose misunderstandings early and reduce costly changes later.

10. Requirement Traceability Matrix (RTM)

A **Requirement Traceability Matrix (RTM)** is a document that maps requirements to their corresponding design components, implementation tasks and test cases.

Its primary purpose is to ensure that every requirement is properly implemented and tested.

A simplified RTM can look like:

Requirement ID	Requirement	Development Component	Test Case	Status
REQ-01	User registration	Authentication module	TC-01	Completed

REQ-02	User login	Authentication module	TC-02	Completed
REQ-03	Search hackathons	Search module	TC-03	In Progress
REQ-04	Filter by technology	Filter module	TC-04	Pending

Benefits of RTM

- Ensures requirement coverage
- Prevents requirements from being forgotten
- Helps testers create corresponding test cases
- Makes impact analysis easier when requirements change
- Improves project visibility
- Supports validation and auditing

The RTM creates a traceable relationship:

Requirement → Design → Implementation → Test → Result

11. Requirement Management Tools

As projects become larger, managing requirements through spreadsheets and documents becomes difficult.

Professional requirement-management tools provide centralized storage, traceability, collaboration, version control and change management.

Examples

Jira

Useful for Agile requirements, issues, tasks, bugs and sprint management.

IBM DOORS / DOORS Next

Designed for structured requirement management and traceability, particularly in large and regulated engineering environments.

Confluence

Useful for maintaining project documentation, specifications and knowledge bases.

The choice of tool depends on project size, industry requirements, compliance needs and development methodology.

12. Case Study: Applying the Concepts to a Software Project

Consider a hackathon aggregation platform where students can discover hackathons from different platforms.

A simplified requirement may be:

Students should be able to search and filter hackathons based on technology, location and deadline.

The requirement can be transformed into a user story:

As a student, I want to filter hackathons by technology and deadline so that I can quickly find suitable opportunities.

This can then be managed through Jira.

Development Flow

Requirement → User Story → Acceptance Criteria → Jira Task → Development → Testing → Deployment → User Feedback → Requirement Improvement

This demonstrates how requirements are connected with the complete software development lifecycle.

13. Agile + DevOps Integration

Agile and DevOps can be combined into a continuous development and delivery cycle.

REQUIREMENTS → BACKLOG → SPRINT → DEVELOPMENT → BUILD → AUTOMATED TEST → DEPLOYMENT → MONITORING → FEEDBACK → BACKLOG UPDATE → NEXT SPRINT

Agile provides the iterative planning and feedback mechanism, while DevOps provides automation and operational feedback.

Together, they support a **continuous software delivery lifecycle**.

14. Key Learnings

Through this self-learning activity, the following concepts were studied:

1. Agile focuses on iterative and adaptive software development.
2. DevOps extends collaboration and automation across development and operations.
3. Jira can be used to manage requirements, tasks, bugs and Agile workflows.
4. User stories express requirements from the user's perspective.
5. Acceptance criteria make requirements measurable and testable.
6. Advanced elicitation techniques help discover both explicit and hidden requirements.
7. RTM provides traceability between requirements, implementation and testing.
8. Requirement-management tools become increasingly important as project complexity increases.
9. Agile and DevOps complement each other rather than replacing one another.

15. Conclusion

Modern software engineering requires organizations to continuously understand user needs, develop software iteratively, validate requirements, and deliver reliable software rapidly.

Agile provides a flexible framework for managing changing requirements and delivering software incrementally, while DevOps connects development with testing, deployment and operations through automation and continuous feedback.

Tools such as Jira support the practical management of this process, while techniques such as user stories, acceptance criteria, advanced requirement elicitation and requirement traceability improve the quality and consistency of requirements.

The overall learning demonstrates that successful software engineering is not limited to programming. It requires a connected process in which **requirements, people, development, testing, deployment and feedback work together as one system.**

16. References

1. AWS – What is DevOps
2. Atlassian Jira Documentation and Agile Resources
3. IBM – Requirements Management and IBM DOORS
4. Agile Manifesto – Principles of Agile Software Development
5. Software Engineering – Ian Sommerville
6. Software Engineering: A Practitioner's Approach – Roger S. Pressman and Bruce R. Maxim